

L'Évaluation de l'Apprentissage

Méthodes d'évaluation des systèmes d'apprentissage automatique

Explication du cours — l'intégralité des diapositives

Notes de lecture, définitions et intuitions

Ce document explique le cours consacré à une question essentielle mais souvent sous-estimée en apprentissage automatique : une fois qu'un modèle est entraîné, comment sait-on s'il est bon ? L'induction — le fait de tirer une règle générale à partir d'exemples particuliers — est une forme de raisonnement intrinsèquement faillible : rien ne garantit qu'une hypothèse qui fonctionne bien sur les données observées fonctionnera aussi bien sur des données nouvelles. Le cours propose donc une méthodologie rigoureuse pour mesurer, comparer et faire confiance (avec prudence) aux performances d'un système d'apprentissage.

1. Pourquoi évaluer ? Les questions de base

Le point de départ du cours est une observation simple mais fondamentale : l'induction est une forme d'inférence faillible. Contrairement à une déduction logique, qui garantit la vérité de sa conclusion si les prémisses sont vraies, une généralisation apprise à partir d'exemples peut toujours se révéler fausse sur de nouveaux cas. Il faut donc absolument savoir évaluer la qualité de ce qu'un système a appris.

Trois questions types motivent ce chapitre :

- Quelle est la performance d'un système sur un type de tâche donné ?
- Est-ce que mon système est meilleur qu'un autre ?
- Comment dois-je régler (les paramètres de) mon système ?

À retenir : ces trois questions structurent implicitement tout le reste du cours — mesurer une performance absolue, comparer deux algorithmes entre eux, et optimiser les réglages (hyperparamètres) d'un même algorithme sont trois problèmes liés mais distincts, qui demandent chacun des précautions méthodologiques particulières.

Deux précautions à garder en tête

- **Les résultats obtenus ne mesurent pas la performance absolue d'un algorithme.** Ils dépendent fortement du jeu de données utilisé pour l'évaluation. C'est le théorème du « no free lunch » (nfl) : il n'existe pas de meilleur algorithme dans l'absolu, valable pour tous les problèmes — un algorithme excellent sur un type de données peut être médiocre sur un autre.
- **Le taux d'erreur n'est qu'un critère d'évaluation parmi d'autres.** C'est le critère le plus utilisé pour comparer des algorithmes, mais s'y limiter peut nous « aveugler » et nous tromper sur le type d'algorithme à privilégier ou à inventer (le cours cite à ce sujet les travaux de Léon Bottou et Jean-Louis Dessalles). D'autres critères — intelligibilité des résultats, coûts de calcul, coûts humains — comptent tout autant selon le contexte applicatif.

2. Le cadre général : facteurs contrôlables et incontrôlables

Le cours propose une image simple mais très utile pour organiser toute la réflexion sur l'évaluation : on peut voir un système d'apprentissage comme une boîte noire qui transforme des entrées en sorties, sous l'influence de deux types de facteurs distincts.

Facteurs contrôlables

Ce sont les éléments que l'expérimentateur choisit et maîtrise directement :

- **L'algorithme** lui-même (quel modèle d'apprentissage utiliser).
- **Ses méta-paramètres (hyperparamètres)**, par exemple l'architecture d'un réseau de neurones, le nombre de voisins et la distance utilisée pour un k-plus-proches-voisins (k-ppv), ou le type de prétraitements appliqués aux données.

Facteurs incontrôlables

Ce sont des sources de variabilité qui échappent, au moins en partie, au contrôle de l'expérimentateur :

- **Le bruit dans les données.**
- **Le tirage aléatoire des données** (quels exemples précis se retrouvent dans l'échantillon d'apprentissage, de validation, de test).
- **Le caractère éventuellement aléatoire de l'algorithme lui-même**, par exemple l'initialisation aléatoire des poids d'un réseau de neurones.

Conséquence méthodologique essentielle : puisque les facteurs incontrôlables introduisent de la variabilité dans les résultats, il faut répéter les expériences pour obtenir une distribution des performances observées, plutôt que de se fier à une seule mesure isolée qui pourrait n'être due qu'à un tirage de données ou une initialisation particulièrement favorable (ou défavorable).

L'optimisation des facteurs contrôlables

Une fois qu'on a identifié les hyperparamètres à régler, plusieurs stratégies existent pour les optimiser :

- Par recherche exhaustive sur une grille (grid search) : on teste systématiquement toutes les combinaisons de valeurs possibles.
- Par optimisation bayésienne : on modélise la fonction de performance en fonction des hyperparamètres et on choisit intelligemment les prochains points à tester.
- Par optimisation par gradient (quand c'est possible, c'est-à-dire quand la performance est différentiable par rapport aux hyperparamètres).
- Par algorithmes évolutionnaires.

Attention, piège classique : l'optimisation des hyperparamètres est réalisée sur l'ensemble de validation, et est donc elle-même sujette à du sur-ajustement (overfitting) — si l'on essaie trop de combinaisons différentes, on finit par en trouver une qui « colle » bien à l'ensemble de validation par pur hasard, sans que cela reflète une vraie qualité du modèle. C'est pourquoi un ensemble de test séparé, jamais utilisé pour le réglage, reste indispensable (voir section suivante).

3. Que mesurer : la matrice de confusion et ses indicateurs

Pour un problème de classification binaire (deux classes, notées « + » et « - »), toute prédiction d'un modèle se range dans l'une de quatre catégories, selon que la prédiction est correcte ou non et selon la classe réelle concernée. C'est la matrice de confusion :

Réel \ Estimé		Classe réelle +	Classe réelle -
	Estimé +	VP (Vrai Positif)	FP (Faux Positif)
	Estimé -	FN (Faux Négatif)	VN (Vrai Négatif)

- **VP (Vrai Positif)** : le modèle prédit « + » et la vraie classe est « + ».
- **FP (Faux Positif)** : le modèle prédit « + » mais la vraie classe est « - ».
- **FN (Faux Négatif)** : le modèle prédit « - » mais la vraie classe est « + ».
- **VN (Vrai Négatif)** : le modèle prédit « - » et la vraie classe est « - ».

Les indicateurs dérivés

À partir de ces quatre quantités de base, on construit tous les indicateurs de performance usuels. Avec m le nombre total d'exemples :

- **Sensibilité (ou Rappel, ou TP-rate)** = $VP / (VP + FN)$: parmi tous les exemples réellement positifs, quelle proportion le modèle a-t-il correctement identifiée ?
- **Spécificité (ou TN-rate)** = $VN / (VN + FP)$: parmi tous les exemples réellement négatifs, quelle proportion le modèle a-t-il correctement identifiée ?
- **Précision** = $VP / (VP + FP)$: parmi tous les exemples que le modèle a annoncés comme positifs, quelle proportion l'était réellement ?
- **Taux d'erreur** = $(FP + FN) / m$, et **Accuracy (exactitude)** = $1 - \text{Taux d'erreur}$: la proportion globale de bonnes (ou de mauvaises) prédictions, tous types confondus.
- **FP-rate** = $FP / (FP + VN)$, et **FN-rate** = $FN / (VP + FN)$: les compléments respectifs de la spécificité et de la sensibilité.

Point clé à comprendre : sensibilité (rappel) et précision répondent à deux questions très différentes. La sensibilité demande « parmi ce qui est vraiment positif, combien ai-je retrouvé ? », la précision demande « parmi ce que j'ai annoncé positif, combien l'était vraiment ? ». Un modèle peut avoir une excellente sensibilité (il retrouve presque tous les positifs) tout en ayant une mauvaise précision (il déclenche aussi beaucoup de fausses alertes), et réciproquement.

La F-mesure : combiner précision et rappel

Comme précision et rappel peuvent évoluer en sens opposé (augmenter l'un se fait souvent au détriment de l'autre), on les combine parfois en un seul indicateur, la F-mesure, une moyenne harmonique pondérée des deux :

$$F\text{-measure} = (\beta^2 + 1) \times \text{Precision} \times \text{Recall} / (\beta^2 \times \text{Precision} + \text{Recall})$$

Le cas particulier le plus utilisé est le F1-score, où $\beta = 1$, qui donne un poids égal à la précision et au rappel :

$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall}) \quad (\text{moyenne harmonique de la précision et du rappel})$$

La raison d'utiliser une moyenne harmonique plutôt qu'une simple moyenne arithmétique : elle pénalise fortement les cas où l'une des deux quantités (précision ou rappel) est très faible — un modèle avec 100 % de précision mais 1 % de rappel obtient un F1 proche de 2 %, pas de 50 %, ce qui reflète mieux l'inutilité pratique d'un tel modèle.

Pourquoi l'accuracy seule peut être trompeuse : un exemple concret

Le cours illustre ce piège avec un exemple réel de comparaison entre deux classifieurs (SVM et Naive Bayes) sur un ensemble de test de 165 exemples (105 « good », 60 « bad ») :

Classifieur	Accuracy	VP (good→good)	FP (bad→good)	FN (good→bad)	VN (bad→bad)
SVM	70.1 %	94	37	11	23
Naive Bayes	62.4 %	72	29	33	31

À première vue, le SVM semble nettement meilleur (70.1 % contre 62.4 % d'accuracy — l'« apparent best »). Mais un examen plus attentif de la matrice de confusion révèle un problème :

- **Le SVM classe 131 exemples comme « good » (94 + 37) contre seulement 34 comme « bad », alors que le test set n'en contient que 105 « good » pour 60 « bad ».** Le SVM est donc biaisé vers la classe majoritaire : il « préfère » prédire « good » quand il hésite, ce qui gonfle artificiellement son score d'accuracy sans que cela reflète une réelle compréhension du problème.
- **Le Naive Bayes, lui, ne classe que 101 exemples comme « good » contre 64 comme « bad », une répartition plus proche de la réalité (105/60).** Si la classe « bad » est celle qui importe vraiment pour l'application (détecter les mauvais cas, par exemple un défaut de paiement), le Naive Bayes — pourtant moins bon en accuracy globale — pourrait être le choix le plus pertinent.

Leçon à retenir : l'accuracy globale masque complètement la façon dont les erreurs se répartissent entre les classes. Sur un jeu de données déséquilibré (une classe plus fréquente que l'autre), un modèle peut afficher une bonne accuracy simplement en favorisant la classe majoritaire — d'où l'importance de toujours examiner la matrice de confusion complète, et pas seulement le taux de bonnes réponses global.

4. Comment mesurer : ensembles de données et protocoles

Le découpage des données

La pratique standard consiste à découper les données disponibles en trois ensembles distincts, chacun avec un rôle précis :

- **Ensemble d'apprentissage (learning/training set)** : sert à entraîner le modèle, c'est-à-dire à ajuster ses paramètres internes (les poids, par exemple).
- **Ensemble de validation (validation set)** : sert à régler les hyperparamètres (méta-paramètres) du modèle, en choisissant la configuration qui minimise l'erreur sur cet ensemble.
- **Ensemble de test (test set)** : sert uniquement, à la toute fin, à estimer la performance réelle du modèle final sur des données jamais vues ni pour l'apprentissage, ni pour le réglage des hyperparamètres.

Règle d'or : chaque ensemble ne doit servir qu'à un seul rôle. Utiliser l'ensemble de test pour régler quoi que ce soit (même indirectement) invalide son rôle d'estimateur non biaisé de la performance réelle — c'est une erreur méthodologique fréquente et grave.

Le cas idéal : la prédiction asymptotique

Quand on dispose d'énormément de données, un phénomène simple et rassurant s'observe : à mesure que le nombre d'exemples d'apprentissage n augmente, l'erreur sur l'ensemble de test (E_{test}) diminue et converge vers une valeur limite a , tandis que l'erreur sur l'ensemble d'apprentissage (E_{train}) — initialement très basse car le modèle « connaît » ses propres exemples — remonte légèrement pour converger vers cette même valeur a . Les deux courbes se rejoignent : le modèle généralise alors aussi bien qu'il mémorise. C'est le comportement attendu pour de très grands jeux de données.

Le sur-apprentissage (overfitting) au fil de l'entraînement

Dans la pratique, surtout avec des données limitées, un phénomène différent et bien plus fréquent s'observe pendant l'entraînement lui-même : l'erreur sur la base d'apprentissage diminue continûment, mais l'erreur sur la base de test (ou de validation) diminue d'abord, atteint un minimum, puis recommence à augmenter.

Conséquence pratique — l'arrêt précoce : on doit arrêter l'apprentissage au moment où l'erreur de validation est minimale, pas au moment où l'erreur d'apprentissage est minimale. Continuer au-delà de ce point fait entrer le modèle en zone de sur-apprentissage : il continue de s'améliorer sur les exemples déjà vus mais se dégrade sur les exemples nouveaux.

Le cours illustre ce phénomène par la question ouverte : « Courbes pour 1 000 exemples ; qu'en serait-il pour 2 000 exemples ? » — suggérant que la quantité de données disponible influence directement le moment où le sur-apprentissage apparaît (plus de données retarde généralement ce phénomène).

Formaliser l'erreur : risque réel et risque empirique

Le cours introduit une distinction fondamentale en théorie de l'apprentissage, entre ce que l'on voudrait mesurer et ce que l'on peut réellement mesurer :

- **L'erreur vraie (ou risque réel)** est une intégrale sur toutes les données possibles D (connues et inconnues), pondérée par leur probabilité d'occurrence $p(x,y)$ — une quantité par définition inaccessible en pratique, puisqu'on ne dispose jamais de « toutes les données possibles ».

$$e_D = \int_D |y - f(x, \theta)| p(x, y) dx, y$$

- **L'erreur de test (ou risque empirique)** est la moyenne de l'erreur observée sur un échantillon fini T de m exemples de test — la seule quantité réellement calculable, qui sert d'estimateur (imparfait) de l'erreur vraie.

$$\hat{e}_S = (1/m) \sum_{(x,y) \in T} |y - f(x, \theta)|$$

L'enjeu central de tout ce chapitre : comme on ne peut jamais mesurer directement l'erreur vraie, toute la méthodologie d'évaluation vise à estimer, aussi fidèlement que possible, à quel point l'erreur de test observée sur un échantillon fini est représentative de l'erreur vraie — et avec quelle marge d'incertitude.

5. Les intervalles de confiance : quantifier l'incertitude

Puisque l'erreur mesurée sur un ensemble de test fini n'est qu'une estimation, il est essentiel de l'accompagner d'une marge d'incertitude : c'est le rôle de l'intervalle de confiance.

Définition

Définition : un intervalle de confiance à N % pour une variable p est l'intervalle dans lequel sa valeur est attendue avec une probabilité de N %.

Le modèle probabiliste sous-jacent : la loi binomiale

Si l'on considère chaque exemple de test comme un « tirage » indépendant, avec une probabilité p de se tromper, le nombre d'erreurs r observées sur n exemples suit une loi binomiale :

$$P(r) = [n! / (r! (n-r)!)] \times p^r \times (1-p)^{(n-r)}$$

Cette loi a pour espérance $E[X] = np$, pour variance $\text{Var}(X) = np(1-p)$, et pour écart-type $\sigma_X = \sqrt{np(1-p)}$.

L'approximation par la loi normale

En pratique, calculer directement avec la loi binomiale est lourd. Heureusement, dès que $np(1-p) \geq 5$, la loi binomiale peut être approchée par une loi normale de même moyenne et même variance :

$$p(x) = 1/\sqrt{2\pi\sigma^2} \times e^{-(1/2)((x-\mu)/\sigma)^2}$$

On applique alors ce résultat à notre problème : l'erreur mesurée sur le test (erreur_T) est elle-même vue comme une variable aléatoire, dont on veut faire dériver une estimation de l'erreur vraie (erreur_D) :

$$\sigma_{T(h)} \approx \sqrt{[\text{erreur}_T(h) \times (1-\text{erreur}_T(h)) / n]}$$

La formule pratique de l'intervalle de confiance

Avec une probabilité de N %, l'erreur vraie erreur_D se trouve dans l'intervalle :

$$\text{erreur}_T(h) \pm z_N \times \sqrt{[\text{erreur}_T(h) \times (1-\text{erreur}_T(h)) / n]}$$

où z_N est un coefficient qui dépend du niveau de confiance souhaité :

N %	50%	68%	80%	90%	95%	98%	99%
z_N	0.67	1.0	1.28	1.64	1.96	2.33	2.58

Une condition de validité (cf. Mitchell, 1997) est généralement retenue : l'ensemble de test T doit contenir m exemples tirés indépendamment, avec $m \geq 30$. Dans ce cas, avec une probabilité de 95 %, l'erreur vraie e_D se trouve dans l'intervalle :

$$\hat{e}_S \pm 1.96 \times \sqrt{\hat{e}_S (1-\hat{e}_S) / m}$$

Exemple travaillé

Une hypothèse classe mal 12 des 40 exemples de la base de test T. Quelle sera l'erreur vraie sur des exemples non vus ?

- $m = 40$, donc $\hat{e}_S = 12/40 = 0.3$
- Le terme d'incertitude : $1.96 \times \sqrt{[0.3 \times 0.7 / 40]} \approx 0.14$

Réponse : avec 95 % de confiance, l'erreur vraie se trouve dans l'intervalle [0.16 ; 0.44]. Cet intervalle, relativement large, illustre bien pourquoi un petit échantillon de test (ici 40 exemples) donne une estimation assez peu précise de la performance réelle du modèle : un taux d'erreur mesuré de 30 % peut correspondre, en vérité, à une erreur réelle allant de 16 % à 44 %.

Le cours illustre également, sur un graphique d'intervalles de confiance à 95 %, comment cet intervalle se resserre à mesure que le nombre d'exemples de test augmente (des courbes distinctes sont tracées pour $n = 10, 15, 30, 50, 100, 250, 1000\dots$) : plus on a de données de test, plus l'estimation de l'erreur vraie devient précise.

Comparer deux hypothèses (deux modèles) entre eux

Le même raisonnement s'étend à la comparaison de deux modèles θ_1 et θ_2 . On cherche la différence vraie de performance :

$$d = e_D(\theta_1) - e_D(\theta_2)$$

qu'on estime par la différence des erreurs empiriques :

$$\hat{d} = \hat{e}_S(\theta_1) - \hat{e}_S(\theta_2)$$

Cette différence suit elle-même (approximativement) une loi normale — la différence de deux lois normales étant encore normale — ce qui donne l'intervalle de confiance à 95 % :

$$\hat{d} \pm 1.96 \times \sqrt{[\hat{e}_S(\theta_1)(1-\hat{e}_S(\theta_1))/m_1 + \hat{e}_S(\theta_2)(1-\hat{e}_S(\theta_2))/m_2]}$$

Remarque importante du cours : il faudrait normalement ne pas tester les deux hypothèses sur le même ensemble de test (l'indépendance supposée par la formule serait alors violée). En pratique, tester les deux modèles sur le même ensemble de test donne cependant une estimation de la variance un peu plus fiable — on parle alors de tests appariés (paired t-tests), une approche statistique légèrement différente mais mieux adaptée à cette situation très courante.

6. Que faire quand on manque de données ?

Toute la méthodologie des intervalles de confiance décrite ci-dessus suppose qu'on peut se permettre de « sacrifier » une bonne partie des données pour constituer des ensembles de validation et de test suffisamment grands. Or ce n'est pas toujours possible.

Le problème du découpage simple (hold-out) avec peu de données

Imaginons que 200 exemples seulement soient disponibles pour l'entraînement :

- Un découpage 50/50 sous-estime l'exactitude de généralisation : la moitié des données étant réservée au test, le modèle entraîné sur l'autre moitié seulement est nécessairement moins bon que celui qu'on obtiendrait avec toutes les données — on mesure donc la performance d'un modèle sous-entraîné, pas celle du modèle final.
- Un découpage 80/20 (davantage de données pour l'apprentissage) donne une estimation basée sur un tout petit échantillon de test (40 exemples ici) — comme on l'a vu, cela conduit à des intervalles de confiance larges et peu fiables, et des tirages différents de cet échantillon de test (different hold-out sets) peuvent donner des résultats très différents.

Le dilemme : plus on garde de données pour l'apprentissage, moins l'estimation de l'erreur (sur un petit ensemble de test) est fiable ; plus on garde de données pour le test, moins le modèle entraîné (sur un petit ensemble d'apprentissage) est représentatif du modèle final. Les techniques suivantes visent à résoudre ce dilemme.

La validation croisée à k plis (k-fold cross-validation)

L'idée : diviser les données en k parties (plis) de taille égale. On répète alors k fois l'opération suivante : entraîner le modèle sur $k-1$ des plis (en jaune), et le tester sur le pli restant (en rose). Chaque pli sert ainsi une fois, et une seule, de test. L'erreur finale est la moyenne des k erreurs obtenues :

$$\text{erreur} = (1/k) \sum_i \text{erreur}_i$$

Cette approche permet d'utiliser toutes les données à la fois pour l'apprentissage et pour le test (à des moments différents), sans jamais mélanger les deux rôles pour un même exemple lors d'une même itération.

Le cas extrême : le leave-one-out

C'est un cas particulier de validation croisée à k plis, où k est égal au nombre total d'exemples n : à chaque itération, un seul exemple sert de test, tous les autres servant à l'apprentissage.

- **Faible biais** : puisqu'on entraîne sur presque toutes les données à chaque fois ($n-1$ exemples sur n), l'estimation de la performance est très proche de celle qu'on obtiendrait avec le modèle final entraîné sur toutes les données.
- **Haute variance** : chaque test individuel ne porte que sur un seul exemple, donc chaque erreur individuelle est très instable (0 ou 1).
- **Tend à sous-estimer l'erreur si les données ne sont pas vraiment i.i.d.** (indépendantes et identiquement distribuées) — par exemple si des exemples très similaires ou corrélés apparaissent plusieurs fois dans le jeu de données (référence : Guyon & Elisseeff, JMLR, 2003).

Le bootstrap

Une troisième approche : au lieu de découper les données sans remise (comme dans le k-fold), on tire aléatoirement, avec remise, un échantillon de la même taille que les données d'origine pour constituer l'ensemble d'apprentissage (certains exemples peuvent donc apparaître plusieurs fois, d'autres pas du tout). Les exemples non tirés constituent l'ensemble de test. On répète l'opération plusieurs fois et on fait la moyenne des erreurs obtenues.

Une propriété remarquable du bootstrap est utilisée pour combiner de façon pondérée l'estimation du risque obtenue sur les échantillons de test successifs ($\bar{P}_1$) avec celle obtenue en ré-évaluant le modèle final sur l'ensemble complet des données ($\bar{P}_2$) :

$$\hat{R}_{\text{Réal}}(h) = 0.636 \times \bar{P}_1 + 0.368 \times \bar{P}_2$$

Un problème méthodologique commun à toutes ces techniques : le calcul classique des intervalles de confiance (section précédente) suppose l'indépendance des estimations successives. Or, dans le k-fold, le leave-one-out ou le bootstrap, les différentes estimations d'erreur partagent en grande partie les mêmes données d'apprentissage d'une itération à l'autre — elles sont donc statistiquement dépendantes. Les intervalles de confiance calculés « naïvement » à partir de ces techniques ont donc tendance à être trop optimistes (trop étroits) par rapport à la réalité.

7. La courbe ROC : arbitrer entre deux types d'erreurs

Deux types d'erreurs à distinguer

- **Erreur de type 1 (α) — les faux positifs** : la probabilité d'accepter l'hypothèse (par exemple, prédire la classe positive) alors qu'elle est fausse.
- **Erreur de type 2 (β) — les faux négatifs** : la probabilité de rejeter l'hypothèse alors qu'elle est vraie.

La question centrale posée par le cours : comment arbitrer entre ces deux types d'erreurs, sachant qu'un modèle de classification produit généralement un score continu (une probabilité, par exemple), et que c'est le choix d'un seuil de décision T qui détermine, à partir de ce score, si l'on prédit finalement la classe positive ou négative ?

Visualiser le compromis : les distributions de scores par classe

Imaginons que le classifieur attribue un score à chaque exemple, et que ce score suive une certaine distribution selon la vraie classe de l'exemple : les exemples de la classe « + » ont tendance à recevoir des scores plus élevés, les exemples de la classe « - » des scores plus bas — mais avec un certain recouvrement entre les deux distributions (aucun classifieur n'est parfait).

Si l'on fixe un seuil de décision T sur ce score :

- Les exemples de la classe « + » dont le score dépasse T sont des vrais positifs ; ceux en-dessous sont des faux négatifs.
- Les exemples de la classe « - » dont le score dépasse T sont des faux positifs ; ceux en-dessous sont des vrais négatifs.

Le compromis fondamental : déplacer le seuil T change simultanément les deux taux d'erreur, mais en sens opposé. Un seuil plus « sévère » (T élevé) réduit les faux positifs mais augmente les faux négatifs. Un seuil plus « laxiste » (T bas) fait l'inverse. Il n'existe pas de seuil qui minimise les deux types d'erreurs à la fois — le choix dépend du coût relatif de chaque type d'erreur pour l'application visée (par exemple, en diagnostic médical, un faux négatif — ne pas détecter une maladie présente — est souvent bien plus coûteux qu'un faux positif).

Définition et construction de la courbe ROC

Définition : ROC signifie Receiver Operating Characteristic (caractéristique de fonctionnement du récepteur, terminologie héritée du traitement du signal). La courbe ROC trace, pour toutes les valeurs possibles du seuil de décision T (de 0 à 1), le taux de vrais positifs (TPR, en ordonnée) en fonction du taux de faux positifs (FPR, en abscisse).

Concrètement, la construction se fait ainsi : on trie tous les exemples de test par score décroissant, puis on fait varier le seuil T de la valeur maximale à la valeur minimale. À chaque valeur de T , on calcule le FPR et le TPR cumulés sur l'ensemble des exemples déjà « au-dessus » du seuil. Chaque exemple correctement classé comme positif (à mesure que le seuil baisse) fait progresser la courbe verticalement (TPR augmente) ; chaque exemple incorrectement classé comme positif la fait progresser horizontalement (FPR augmente). On obtient ainsi une courbe en escalier qui part du point (0,0) — seuil maximal, aucune prédiction positive — et arrive au point (1,1) — seuil minimal, toutes les prédictions sont positives.

Interpréter la courbe ROC

- **La diagonale (« ligne de hasard »)** représente un classifieur qui ne fait pas mieux qu'un tirage aléatoire ($TPR = FPR$ quel que soit le seuil) — une pertinence de 0,5.
- **Plus la courbe ROC s'écarte de la diagonale vers le coin supérieur gauche, meilleur est le classifieur.**
Un point idéal serait (0,1) : aucun faux positif, tous les vrais positifs détectés.
- **Un point unique sur le graphique** (par exemple les points P, SVM, NB, Q vus dans l'exemple du cours) représente la performance d'un classifieur pour un seuil de décision fixé, tandis que la courbe entière représente toutes les performances possibles de ce même classifieur en faisant varier le seuil.

La courbe ROC permet ainsi de comparer deux classifieurs indépendamment du choix d'un seuil particulier : un classifieur dont la courbe ROC domine entièrement celle d'un autre (elle est toujours au-dessus, quel que soit le FPR) est objectivement meilleur, quel que soit le compromis faux positifs / faux négatifs finalement recherché par l'utilisateur.

8. Autres mesures et critères d'évaluation

La matrice de confusion multi-classes

Quand le problème comporte plus de deux classes, la matrice de confusion se généralise naturellement : chaque ligne représente la classe réelle, chaque colonne la classe prédite, et chaque cellule (i,j) indique la proportion d'exemples de la classe réelle i qui ont été (mal ou bien) classés comme appartenant à la classe j . La diagonale ($i = j$) indique les bonnes classifications ; tout le reste indique des confusions entre classes.

Le cours illustre cela avec un exemple de classification d'espèces animales (oiseaux, poissons, papillons...) : 14 % des papillons, par exemple, sont pris pour des poissons par le modèle — une information précieuse pour comprendre quelles classes le modèle confond entre elles, et donc où concentrer les efforts d'amélioration (plus de données pour ces classes, de meilleurs descripteurs, etc.).

Au-delà de la performance : d'autres critères d'évaluation

Le cours insiste, en écho à sa mise en garde initiale, sur le fait que le taux d'erreur (ou toute mesure dérivée de la matrice de confusion) n'est qu'un critère d'évaluation parmi d'autres, à mettre en balance avec :

- **L'intelligibilité des résultats (des hypothèses produites)** : un modèle dont on peut comprendre et expliquer les décisions (par exemple un arbre de décision) peut être préféré à un modèle plus performant mais totalement opaque (le cours cite avec un clin d'œil : « exit les réseaux de neurones » dans ce contexte particulier où l'interprétabilité prime).
- **Les performances en généralisation**, qui ne sont pas toujours en adéquation totale avec l'intelligibilité — un modèle simple et interprétable n'est pas toujours celui qui généralise le mieux, et inversement.
- **Les coûts**, sous plusieurs formes : le coût de préparation des données, le coût computationnel (coût d'une passe d'apprentissage et nombre de passes nécessaires), le coût de l'expertise en apprentissage automatique nécessaire pour mettre en œuvre et régler la méthode, et le coût de l'expertise sur le domaine d'application nécessaire pour interpréter correctement les résultats.

9. Résumé : ce qu'il faut retenir

1. **Soignez votre fonction de coût** : demandez-vous toujours ce qui importe réellement pour votre mesure de performance (accuracy globale, sensibilité, précision, F-mesure, coûts asymétriques entre types d'erreurs...) plutôt que de vous fier par défaut au taux d'erreur brut.
2. **Avec des données en nombre fini, calculez les intervalles de confiance** : une erreur mesurée sans intervalle de confiance associé ne veut pas dire grand-chose, surtout sur un petit échantillon de test.
3. **Avec des données rares, faites attention à la répartition entre apprentissage et test, et recourez à la validation croisée** : le k-fold, le leave-one-out ou le bootstrap permettent d'utiliser au mieux des données limitées, chacun avec ses propres compromis biais/variance.
4. **N'oubliez jamais l'ensemble de test**, distinct de l'ensemble de validation utilisé pour le réglage des hyperparamètres — sans quoi l'estimation finale de la performance est biaisée de façon optimiste.
5. **L'évaluation est une étape très importante** : gardez l'esprit critique face à des résultats qui semblent trop beaux, et efforcez-vous de vous convaincre vous-même, par des vérifications rigoureuses, avant de convaincre les autres.

Références bibliographiques

Auteurs	Titre	Éditeur
Nathalie Japkowicz & Mohak Shah	<i>Evaluating Learning Algorithms. A classification perspective.</i>	Cambridge University Press, 2011

Document préparé à partir du cours « Évaluation de l'apprentissage : méthodes » (A. Cornuéjols, AgroParisTech — basé sur Sebastian Thrun, CMU, et sur le tutoriel de Padraic Cunningham, ECML-09).